

MARAN PMS — Prompt 46: Grupos — Fixes Críticos + UX

Archivos principales

- `client/src/pages/groups.tsx` — `GroupsPage` + `GroupFormDialog`
 - `client/src/pages/group-detail.tsx` — `GroupDetailPage` + `AssignBlockDialog`
 - `client/src/pages/planning.tsx` — aviso de bloqueos sin asignar
 - `server/routes.ts` — endpoint `/api/groups/:id/payment`
 - `server/storage.ts` — `assignRoomToGroup`
 - `shared/schema.ts` — schema de groups
-

SECCIÓN 1 — ERRORES CRÍTICOS

1.1 Fix: fechas que se modifican solas (bug UTC)

Causa raíz: `GroupFormDialog` en `client/src/pages/groups.tsx` usa:

```
const today = new Date().toISOString().split("T")[0];
const tomorrow = new Date(Date.now() + 86400000).toISOString().split("T")[0];
```

`toISOString()` devuelve UTC. En Argentina (UTC-3), a las 21hs del día 12 el ISO devuelve "13". Cada vez que se abre el dialog el `useEffect` resetea el form con este `today` incorrecto, sobrescribiendo las fechas guardadas.

Fix — reemplazar ambas líneas en `GroupFormDialog` :

```
// ANTES:
const today = new Date().toISOString().split("T")[0];
const tomorrow = new Date(Date.now() + 86400000).toISOString().split("T")[0];

// DESPUÉS:
const today = new Date().toLocaleDateString("en-CA", { timeZone: "America/Argenti
```

```
const tomorrowDate = new Date();
tomorrowDate.setDate(tomorrowDate.getDate() + 1);
const tomorrow = tomorrowDate.toLocaleDateString("en-CA", { timeZone: "America/Ar
```

Este fix aplica tanto al `useState` inicial como al `useEffect` que resetea el form (ambos usan estas mismas variables).

1.2 Fix: editar reserva no debe desasignarla del planning

Causa raíz: el `ReservationEditForm` (en `client/src/pages/reservations.tsx` o componente compartido) tiene:

```
const availableRooms = rooms.filter(r =>
  r.status === "available" && r.roomTypeId === selectedRoomTypeId
);
```

Si la habitación actual está `occupied` (por la misma reserva), NO aparece en el selector. El usuario abre el form, el selector queda vacío, y al guardar se envía `roomId: ""`.

Fix 1 — incluir siempre la habitación actual:

```
const availableRooms = rooms.filter(r =>
  (r.status === "available" || r.id === reservation?.roomId) &&
  r.roomTypeId === selectedRoomTypeId
);
```

(Verificar que ya esté así — si está, el problema es el fix 2.)

Fix 2 — nunca enviar roomId vacío en el PATCH:

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  const payload = {
    ...formData,
    // NUNCA enviar roomId vacío — preservar el existente
    roomId: formData.roomId || reservation?.roomId,
    // NUNCA enviar roomTypeId vacío
    roomTypeId: formData.roomTypeId || reservation?.roomTypeId,
    bedTypeId: formData.bedTypeId || null,
    nights: Number(formData.nights),
    numberOfGuests: Number(formData.numberOfGuests),
    baseRatePerNight: String(formData.baseRatePerNight || "0"),
    finalRatePerNight: String(formData.finalRatePerNight || "0"),
```

```
totalRoomAmount: String(formData.totalRoomAmount || "0"),
discountValue: String(formData.discountValue || "0"),
};
mutation.mutate(payload);
};
```

Fix 3 — inicializar `selectedRoomTypeId` correctamente al abrir el form: En el `useEffect` que resetea el form al abrir, asegurarse de que `selectedRoomTypeId` se inicialice con `reservation?.roomId` antes de que el render del select intente filtrar rooms.

1.3 Fix: validación de superposición al editar reserva de grupo

El backend ya tiene protección en `PATCH /api/reservations/:id`:

```
const finalRoomId = req.body.roomId || existing.roomId;
// ... checkOverbooking solo si roomIdChanged || datesChanged
```

Esto es correcto. La protección falla solo cuando llega `roomId: ""` desde el frontend (fix 1.2 lo resuelve).

Verificación adicional en `assignRoomToGroup` (`server/storage.ts`): antes de crear la reserva, verificar que no haya conflicto:

```
// Agregar antes del createReservation():
const hasConflict = await this.checkOverbooking(roomId, checkInDate, checkOutDate
if (hasConflict) {
  throw new Error(`La habitación ${room.roomNumber} ya tiene una reserva en esas
}
```

1.4 Pagos grupales: permitir pago con reservas en estado confirmed

Problema: el endpoint `POST /api/groups/:id/payment` en `server/routes.ts` solo procesa reservas `checked_in`, rechazando pagos anticipados:

```
// ACTUAL:
const checkedInReservations = group.reservations.filter(
  r => r.status === "checked_in"
);
if (checkedInReservations.length === 0) {
  return res.status(400).json({ error: "No hay reservas en casa..." });
}
```

```
}
```

Fix:

```
// NUEVO — incluir confirmed y checked_in:
const activeReservations = group.reservations.filter(
  r => r.status === "confirmed" || r.status === "checked_in"
);
if (activeReservations.length === 0) {
  return res.status(400).json({
    error: "No hay reservas activas (confirmadas o en casa) para registrar pagos"
  });
}
// reemplazar checkedInReservations por activeReservations en todo el bloque de d
```

SECCIÓN 2 — MEJORAS UX

2.1 Aviso de bloqueos sin asignar: colapsable

Ubicación: en `client/src/pages/planning.tsx`, la sección que muestra `unassignedGroupBlocks` del response de `/api/planning`.

Fix: envolver la sección en un panel colapsable con estado persistente en `localStorage`:

```
const [blocksCollapsed, setBlocksCollapsed] = useState(() => {
  return localStorage.getItem("planning_blocks_collapsed") === "true";
});

const toggleCollapsed = () => {
  const next = !blocksCollapsed;
  setBlocksCollapsed(next);
  localStorage.setItem("planning_blocks_collapsed", String(next));
};

// Render:
{unassignedGroupBlocks && unassignedGroupBlocks.length > 0 && (
  <div className="border rounded-lg overflow-hidden">
    <button
      onClick={toggleCollapsed}
      className="w-full flex items-center justify-between px-4 py-2 bg-amber-50 d
    >
    <span className="flex items-center gap-2">
```

```

        <AlertTriangle className="h-4 w-4" />
        {unassignedGroupBlocks.length} bloque(s) de grupo sin asignar
      </span>
      <ChevronDown className={`h-4 w-4 transition-transform ${blocksCollapsed ? "
    </button>
    {!blocksCollapsed && (
      <div className="p-3 space-y-1">
        {unassignedGroupBlocks.map((block, i) => (
          <div key={i} className="text-xs text-amber-700 dark:text-amber-300 flex
            <span>{block.groupName} - {block.roomTypeName}</span>
            <span>{block.assigned}/{block.quantity} asignadas</span>
          </div>
        ))}
      </div>
    )}
  </div>
)}

```

2.2 Contador de habitaciones sin asignar en GroupDetailPage

En `client/src/pages/group-detail.tsx`, en la card de "Habitaciones", agregar indicador visual cuando hay pendientes:

```

<CardContent>
  <div className="flex items-baseline gap-2">
    <span className="text-3xl font-bold">{group.assignedRooms}</span>
    <span className="text-muted-foreground">/ {group.totalRooms} asignadas</span>
  </div>
  {group.totalRooms > group.assignedRooms && (
    <div className="mt-2 flex items-center gap-1 text-sm text-amber-600 dark:text
      <AlertTriangle className="h-3.5 w-3.5" />
      <span>{group.totalRooms - group.assignedRooms} habitación(es) sin asignar</
    </div>
  )}
  <div className="mt-2 h-2 w-full rounded-full bg-muted overflow-hidden">
    <div
      className="h-full bg-primary transition-all"
      style={{ width: `${group.totalRooms > 0 ? (group.assignedRooms / group.tota
        />
    </div>
  </CardContent>

```

2.3 Campos de evento en el grupo: salón y horario

El schema `groups` ya tiene `eventDate`. Agregar dos campos más:

Migración DB:

```
ALTER TABLE groups
  ADD COLUMN event_salon text,
  ADD COLUMN event_time text;
```

Schema (`shared/schema.ts`):

```
export const groups = pgTable("groups", {
  // ... campos existentes ...
  eventDate: date("event_date"),
  eventSalon: text("event_salon"),    // ← nuevo
  eventTime: text("event_time"),      // ← nuevo
});
```

Fix en `GroupFormDialog` (`client/src/pages/groups.tsx`): agregar los campos al formulario en la sección de "Evento":

```
// Agregar al formData inicial:
eventSalon: group?.eventSalon || "",
eventTime: group?.eventTime || "",

// Agregar en el formulario (junto al campo eventDate ya existente):
<div className="grid gap-2">
  <Label>Salón del evento</Label>
  <Input
    value={formData.eventSalon || ""}
    onChange={(e) => setFormData({ ...formData, eventSalon: e.target.value })}
    placeholder="Ej: Salón Principal, Terraza..."
    data-testid="input-event-salon"
  />
</div>
<div className="grid gap-2">
  <Label>Horario del evento</Label>
  <Input
    type="time"
    value={formData.eventTime || ""}
    onChange={(e) => setFormData({ ...formData, eventTime: e.target.value })}
    data-testid="input-event-time"
  />
</div>
```

Mostrar en GroupDetailPage: en la card de “Fechas”, agregar:

```
{group.eventDate && (
  <div className="mt-2 pt-2 border-t">
    <p className="text-xs text-muted-foreground uppercase tracking-wide mb-1">Eve
    <p className="text-sm">Fecha: <span className="font-medium">{formatDate(group
    {group.eventSalon && <p className="text-sm">Salón: <span className="font-medi
    {group.eventTime && <p className="text-sm">Horario: <span className="font-med
  </div>
)}
```

Importante: estos campos son informativos para el grupo de habitaciones solamente. NO deben crear ni modificar ningún registro en `events` , `event_rooms` , ni ninguna tabla del módulo de eventos. No hay integración con el planning de eventos.

2.4 Rooming list con info del evento

En `GroupDetailPage` , función `printRoomingList()` , agregar la sección de evento en el HTML generado:

```
// Agregar después del bloque de info-row en el HTML:
const eventSection = (group.eventDate || group.eventSalon || group.eventTime) ? `
  <div style="margin-bottom: 16px; padding: 10px 14px; background: #f9f9f9; borde
    <p style="margin: 0 0 4px 0; font-size: 10px; text-transform: uppercase; colo
    ${group.eventDate ? `<p style="margin: 2px 0; font-size: 12px;">Fecha: <stron
    ${group.eventSalon ? `<p style="margin: 2px 0; font-size: 12px;">Salón: <stro
    ${group.eventTime ? `<p style="margin: 2px 0; font-size: 12px;">Horario: <str
  </div>
  ` : "";

// Insertar eventSection en el HTML antes de la tabla
```

NOTAS TÉCNICAS

- El fix 1.1 (fechas UTC) es el más urgente — afecta todos los grupos creados a partir de cierta hora del día.
- El fix 1.2 (roomId vacío) requiere verificar el componente exacto de edición de reservas que se usa desde el contexto de grupo — puede ser `ReservationEditForm` en `reservations.tsx` o un componente compartido.

- Los campos `eventSalon` y `eventTime` son texto libre. No validar contra las salas existentes en el módulo de eventos.
- El `localStorage` para el estado colapsado del panel de bloqueos es opcional — si hay problemas se puede usar solo `useState` sin persistencia.
- Al hacer la migración de `event_salon` y `event_time`, usar `DEFAULT NULL` para no afectar grupos existentes.